MeterSense AI — BESCOM Smart Meter Intelligence & AT&C Loss Detection

PanIIT AI for Bharat Hackathon — Theme 8 Submission

---

1. Executive Summary

BESCOM (Bangalore Electricity Supply Company) must reduce **Aggregate Technical and Commercial (AT&C)** loss across thousands of 11kV/415V feeders. **MeterSense AI** is a single Next.js operations console that ingests **feeder- and consumer-level smart meter intervals**, runs **isolation-forest** cohort scoring per feeder, a **rule engine** (seven anomaly classes), and **feeder reconciliation** (supplied − ∑consumer kWh) to open explainable `Anomaly` cases with **evidence JSON**, **mock narrative explanations** (`USE_MOCK_AI=true`), and **revenue gap estimates** for prioritisation. A **Tremor** dashboard, **Leaflet** substation and pincode choropleth, and an investigator **status workflow** complete the field-ready story—without any live SCADA integration (synthetic data only).

---

2. Architecture

Layer	Responsibility
Ingestion	MDAS/HES-style smart meter files (simulated) + optional CSV upload stub.
Store	Prisma + SQLite: `Substation` → `Feeder` → `Consumer`; time-series in `FeederReading` and `ConsumerReading`.
Detection	`isolation-forest` npm package; z-score path if tiny cohorts; rules in `lib/detectors/rules.ts`; reconciliation in `feederReconciliation.ts`.
AI (mock)	`lib/ai.ts` — template **explainAnomaly** from evidence + type.
UI	Next.js 15 App Router, Tremor charts, **Tailwind v3** + amber/yellow BESCOM palette, `react-leaflet` maps.

See `docs/diagrams/architecture.png` (Mermaid source: `docs/diagrams/architecture.mmd`).

---

3. Data & Detection

- **Isolation forest** — Feature vector per consumer per 30d window: mean kWh, stdev, zero-day ratio, voltage dropouts, reverse-flow count, mean PF. Trained per feeder cohort.
- **Rules** — `ZERO_CONSUMPTION_LIVE`, `REVERSE_FLOW`, `POWER_FACTOR_DROP`, `VOLTAGE_SAG`, `METER_COMM_LOSS`, `BYPASS_SUSPECTED`, `TAMPER_SUSPECTED` (incl. tamper flag pattern).

- **Reconciliation** — Aggregate feeder readings vs under-reporting share to flag **BYPASS_SUSPECTED** when **loss% > 25%** and **< 50%** of consumers exceed a "meaningful" energy threshold.

---

4. Reproducibility

```
cd theme8-smart-meter
cp .env.example .env
npm install
npx prisma migrate deploy
npm run seed
npm run build
npx tsc --noEmit
npm run dev   # http://localhost:3000
```

- **Mock data** — `scripts/generate-mock-meter-data.ts` (Faker `seed(42)` — 5 substations, 20 feeders, 200 consumers, 30d hourly feeder + 6h consumer data, **injected** anomaly scenarios.
- **Seeding** — `scripts/seed-demo.ts` loads data and **calls the same detection pipeline** as `POST /api/detect/run`.

---

5. Security & Ethics

- **Synthetic data only** — no real RR numbers, no live grid telemetry.
- **No credentials in remotes** — use plain `https://github.com/sridhar7601/bescom-meter-intel.git` and GitHub CLI auth.
- **USE_MOCK_AI** default — no third-party LLM required for the demo.

---

6. Known Limitations (MVP)

- No real MDAS/AMISP integration.
- Isolation-forest is lightweight JS and tuned for the hackathon; production would retrain and calibrate per DISCOM policy.
- Pincode choropleth uses **simplified** GeoJSON envelopes for a subset of pincodes (visual proxy).

---

7. Conclusion

MeterSense AI demonstrates a **credible** BESCOM control-room loop: time-series at feeder and consumer depth, **triple-method** anomaly detection, **AT&C framing** in the UI, **explainable** evidence, and a **geospatial** loss view—ready to be swapped for real **API keys** and **PostgreSQL** when moving past the hackathon.